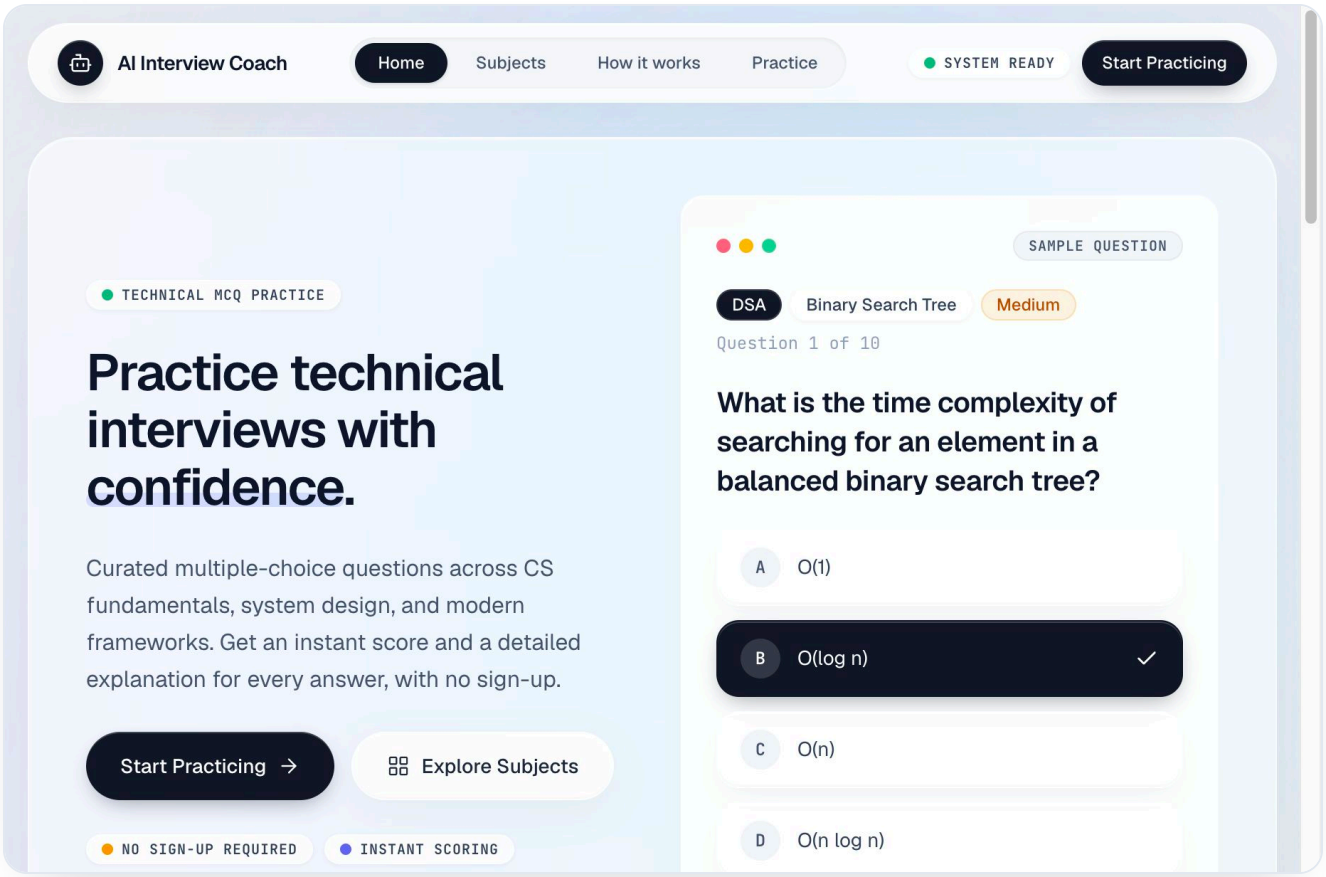
BEGINNER GUIDE · NO PRIOR CURSOR EXPERIENCE NEEDED

9:38 PM I pasted a PRD into Cursor. 10:40 PM it was live on the internet.

This is that website. It's an AI Interview Coach: you enter your name, pick subjects like DSA or React, answer MCQs, and get a score with an explanation for every question. Backend on Render, frontend on Vercel, code on GitHub.



The live site at octeveaiint.vercel.app. Open it on your phone right now.

You guys are going to build the same thing, step by step. This guide assumes you have never opened Cursor. If a step says "click", I tell you what to click. If a word sounds like jargon, it's in the glossary.

GUESS FIRST

At the end of this guide there's a list of every prompt I typed into Cursor. Before you scroll there, guess: how many of those prompts contain actual code?

Write your guess down. We'll check it at the end.

KEEP COUNT

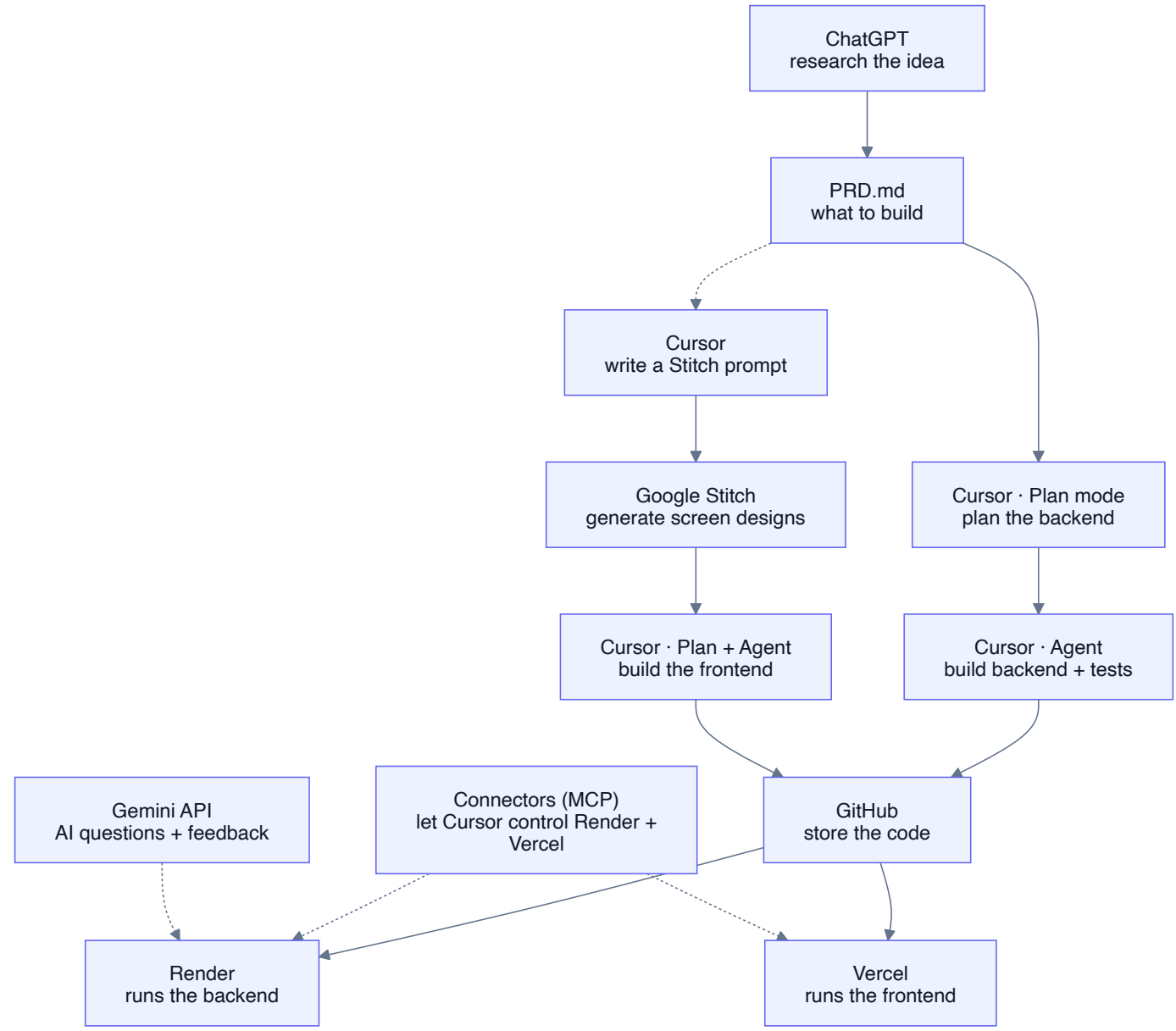
Cursor did almost all the work. But there were exactly **three moments** where it stopped and said "I need you for this one". Keep an eye out for them. I'll reveal all three in Part 8.

THE WHOLE MAP

Six tools, one evening

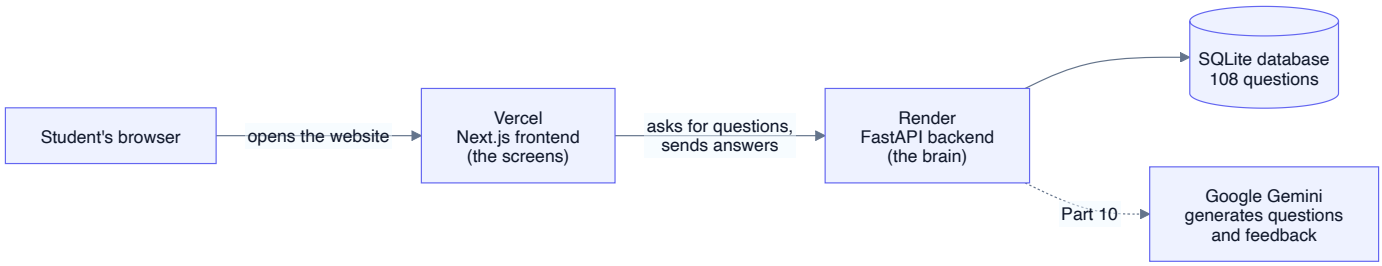
Before we go deep into any one step, look at the whole journey. Every Part in this guide is one box in this diagram. Come back to this map whenever you feel lost.

THE FULL PIPELINE



And here is how the finished app works when a student uses it:

HOW THE LIVE APP WORKS



The real timeline, taken from my Cursor chat history that evening:

FRIDAY EVENING, IST

9:38 PM	PRD attached, backend prompt sent in Plan mode
9:42 PM	Plan approved, clicked Build
9:43 PM	Second chat opened to write the Stitch prompt
9:50 PM	Backend done, 48 tests pass, code pushed to GitHub
9:55 PM	Ran it on my laptop
10:00 PM	Asked about connectors, added Render and Vercel
10:11 PM	Backend live on Render, placeholder live on Vercel
10:21 PM	Gave Cursor the Stitch designs, Plan mode again
10:40 PM	Glassmorphism frontend live
10:43 PM	Started adding Gemini AI

PART 0

Accounts and installs

Do this part before class. It's boring, but every later step depends on it. Budget 45 minutes.

0.1 Create these accounts

All of these have free plans. Use the same email everywhere if you can.

ACCOUNT	WHERE	WHAT IT'S FOR
GitHub	github.com/signup	Stores your code online. Render and Vercel read your code from here.
Cursor	cursor.com	The AI code editor. You'll do 90% of the work here.
ChatGPT	chatgpt.com	Research and writing the PRD.
Google account	stitch.withgoogle.com	Google Stitch generates the screen designs. Also gives you a Gemini API key later.
Render	render.com	Runs your backend on the internet. Sign up with **Continue with GitHub**.
Vercel	vercel.com/signup	Runs your frontend on the internet. Sign up with **Continue with GitHub**.

MY MISTAKE, LEARN FROM IT

My Render account was on one email and my Vercel account on a different one. That caused an extra permission step later (Part 8). If you sign up to both Render and Vercel with **Continue with GitHub**, using the same GitHub account that holds your code, you skip that pain.

0.2 Install the software

Your computer needs five things. Don't worry about what each one does yet; the glossary explains them.

SOFTWARE	MAC	WINDOWS
Cursor	Download from cursor.com/download, open the installer, follow the prompts.	
Git	Open the Terminal app, type `git --version`. If it's missing, macOS offers to install it. Click **Install**.	Download from git-scm.com. Keep all default options.
Node.js 22	Download the installer from nodejs.org. Our frontend needs version 22 or newer. Version 20 broke our build (see What went wrong).	
Python 3.12	Download from python.org/downloads. **Windows:** on the first installer screen, tick **"Add python.exe to PATH"**. People miss this all the time.	
GitHub CLI (`gh`)	Download from cli.github.com, or run `brew install gh` if you have Homebrew.	Download from cli.github.com, or run `winget install --id GitHub.cli`.

0.3 Log the GitHub CLI into your account

This lets Cursor create a GitHub repository and push your code without you touching GitHub's website. Open a terminal (on Mac: the Terminal app; on Windows: PowerShell) and type:

```
gh auth login
```

1. It asks *"Where do you use GitHub?"*. Choose **GitHub.com** with the arrow keys, press `Enter`.
2. *"Preferred protocol"*: choose **HTTPS**.
3. *"Authenticate Git with your GitHub credentials?"*: **Yes**.
4. *"How would you like to authenticate?"*: **Login with a web browser**.
5. It shows a one-time code like `ABCD-1234`. Copy it, press `Enter`, the browser opens, paste the code, click **Authorize**.
6. Back in the terminal you'll see *"Logged in as yourname"*. Done.

CHECK YOURSELF

Once Cursor is installed (Part 1), you can let Cursor check all of this for you. Open the Agent panel and type:

PROMPT

Check whether Git, Node.js, Python and the GitHub CLI are installed on my computer, which versions they are, and whether gh is logged in. Tell me what's missing and how to install it. Don't install anything yourself.

PART 1

Cursor from zero

Cursor is a code editor, like VS Code (it's actually built on VS Code), with an AI agent built in. You talk to the agent in plain English. It reads your files, writes code, runs commands in the terminal, opens a browser to test your app, and tells you what it did.

The mental shift: you're not the typist anymore. You're the person who decides *what* to build and checks that it's right. Cursor does the typing.

1.1 Install and sign in

1. Open the Cursor installer you downloaded in Part 0 and finish the install.
2. Open Cursor. It asks you to **sign in**. Click it, the browser opens, sign in (Google or GitHub is easiest), then come back to Cursor.
3. You may see a few setup screens (theme, shortcuts). Keep the defaults; the exact screens change between versions. If you already use VS Code, you can bring your settings over later from **Cursor Settings → General → Account → VS Code Import**.
4. You'll land on a welcome screen. That's it, Cursor is ready.

PLANS AND MONEY

Cursor's Free plan includes the agent and Tab autocomplete with limited usage and Cursor's own models. A full build like this one may run past the free limit. Paid plans (Plus, Premium and up) give more usage and more models. Cursor's old student discount closed to new sign-ups on 25 June 2026, so don't count on it. Check cursor.com/pricing for current details before you pay for anything.

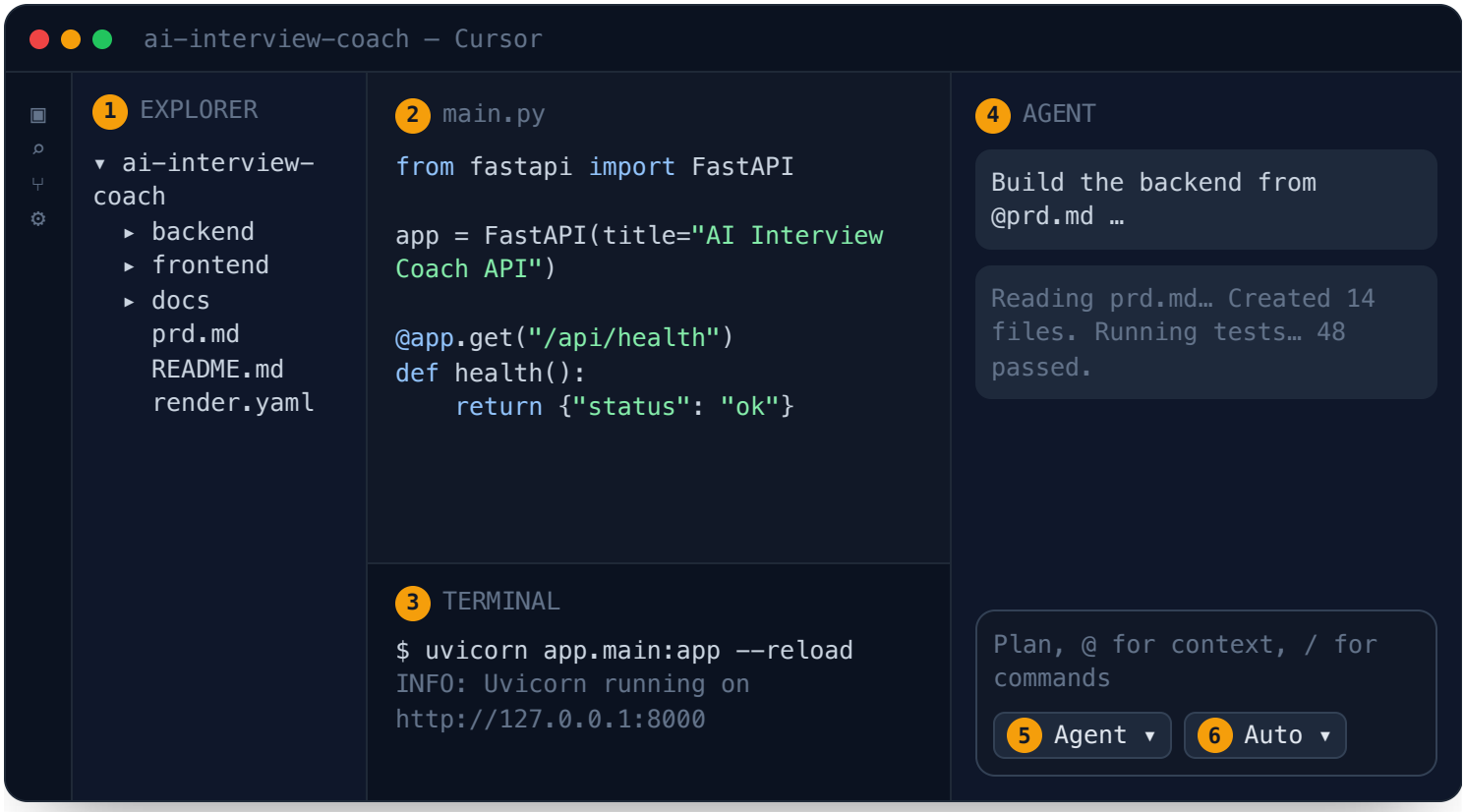
1.2 Make a project folder and open it

Cursor always works inside **one folder**. Everything the agent creates goes inside it. So first make an empty folder.

1. In Finder (Mac) or File Explorer (Windows), go to **Documents** and create a new folder. Name it something with no spaces, like `ai-interview-coach`.
2. In Cursor, click **File → Open Folder...** (on the welcome screen there's also an **Open project** button).
3. Pick your new folder and click **Open**. If Cursor asks *"Do you trust the authors of the files in this folder?"*, click **Yes, I trust the authors**. It's your own empty folder.
4. Drag your `prd.md` file (from Part 2) into this folder. You'll see it appear in the left sidebar.

1.3 A tour of the window

Here's what Cursor looks like with a project open. Learn these six areas and you know enough to start.



1. **Explorer.** Your project's files and folders. Click a file to open it.
2. **Editor.** Where an open file shows. You can read and edit code here.
3. **Terminal.** Where commands run. Open it from the menu: **View → Terminal**.
4. **Agent panel.** Where you talk to Cursor. This is where you'll spend most of your time.
5. **Mode picker.** Agent, Plan, Ask, Debug. Explained in 1.5.

6 **Model picker.** Which AI model answers. Leave it on **Auto**.

The exact look changes between Cursor versions. The agent panel might be on the left or the right, and Cursor 3 added a separate **Agents Window** that shows only chats (open it with `⌘ Cmd` + `Shift` + `P` → *Open Agents Window*; go back with *Open IDE*). For this guide, stay in the normal editor window. The six pieces are always there.

1.4 Talk to the agent

1. Open the Agent panel if it isn't open: `⌘ Cmd` + `I` on Mac, `Ctrl` + `I` on Windows.
2. Click in the text box at the bottom of the panel.
3. Type what you want in normal English. Long, messy, detailed messages are fine. Look at my prompts in this guide: they read like someone talking, typos and all, and they worked. You can also dictate: `⌘ Cmd` + `Shift` + `Space` starts voice input.
4. Press `Enter` to send. (`Shift` + `Enter` adds a new line without sending.)
5. Watch. The agent shows each step: which files it reads, what it edits, which commands it runs. It ends with a summary of what it did.
6. Reply in the same box to continue. Each conversation is called a **chat**. Start a fresh chat with the **+** button at the top of the panel (or `⌘ Cmd` + `N`) when you switch to a new topic.

> **REAL TRICK FROM THAT NIGHT**
>
> You can run **two chats at the same time**. At 9:42 one chat was building my backend. At 9:43 I opened a second chat and asked it to write the Google Stitch prompt. Both worked in parallel. Click **+** for a new chat; your old chats stay in the chat history.

1.5 The four modes

Click the mode picker (number 5 in the diagram) under the text box, or press `Shift` + `Tab` in the chat box to cycle through the modes. You'll see these four. Picking the right one is the single most important Cursor skill.

Agent `does the work`

The default. Reads files, writes and edits code, runs terminal commands, tests things. Use it when you know what you want and want it done.

"Can you run this application for me, I want to test it locally."

Plan `thinks first`

Researches your code, asks you clarifying questions, then writes a step-by-step plan with a to-do list. It does **not** change your code until you click **Build**. Use it for any big feature.

"Build the backend from this PRD." · "Build the frontend from these designs."

Ask `read-only`

Answers questions about your code or a concept. It reads but never changes files. Use it to learn and explore safely.

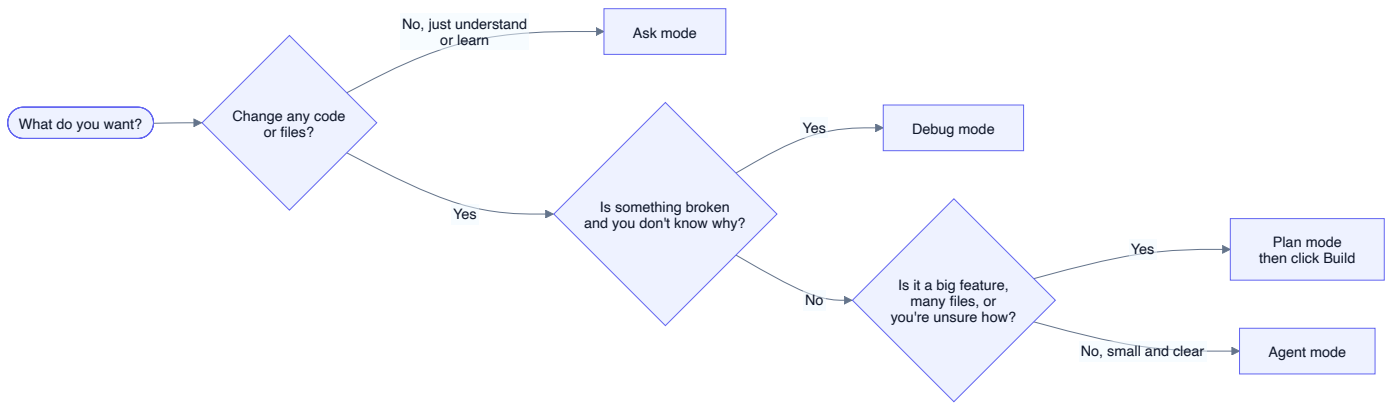
"Explain what render.yaml does, line by line."

Debug `finds the bug`

For bugs you can't figure out. It forms guesses about the cause, adds temporary logging, asks you to reproduce the bug, then fixes it based on what the logs show.

"After I submit, the results page doesn't respond to clicks."

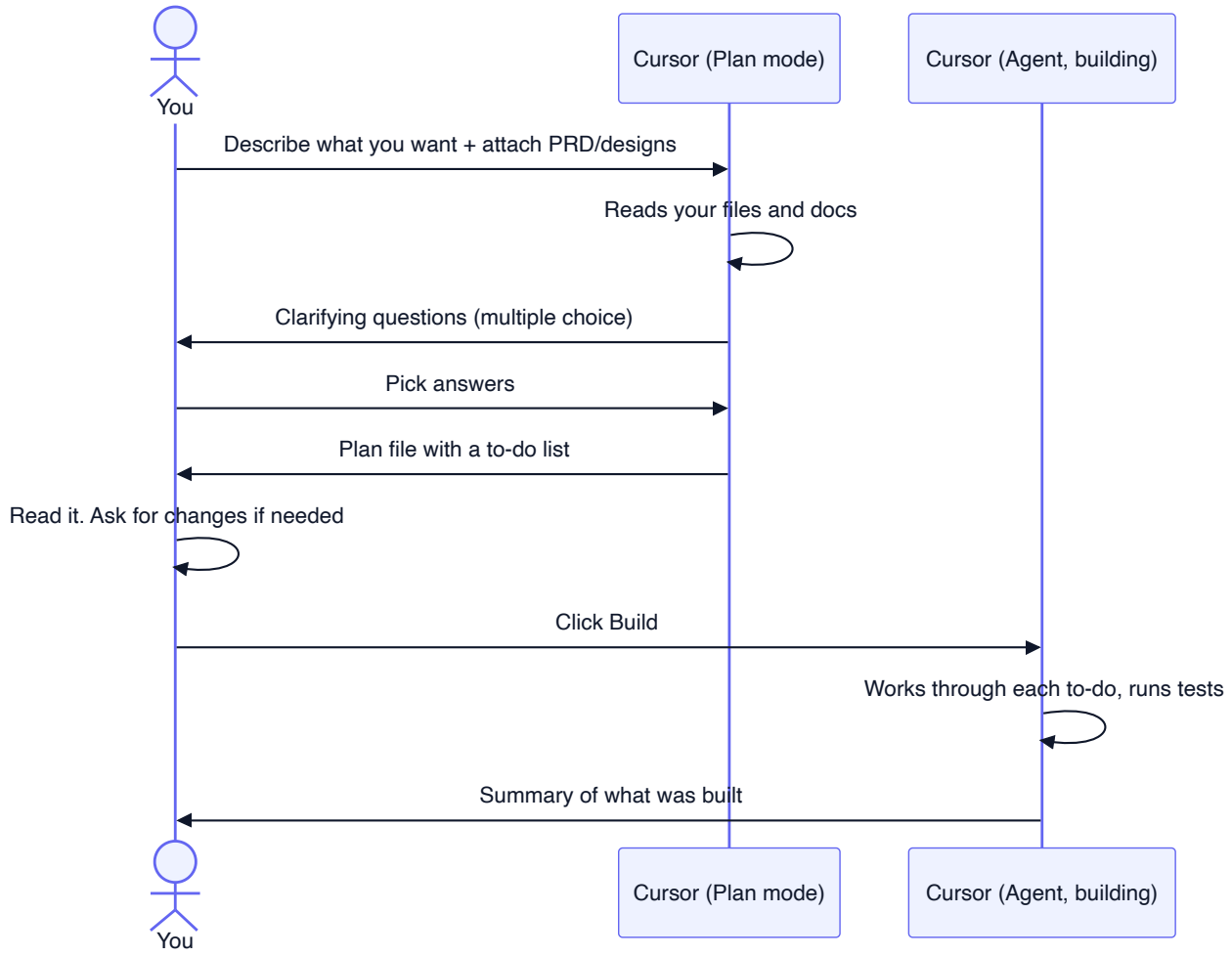
WHICH MODE SHOULD I USE?



1.6 Plan mode, step by step

I used Plan mode for every big step that night: backend, frontend and AI. Here is exactly what happens:

THE PLAN → BUILD LOOP



1. Switch the mode picker to **Plan**.
2. Type your request and attach context (see 1.7). Send it.
3. Cursor reads around, then shows **clickable questions**. These are real decisions, not small talk. For my backend it asked three: free or paid Render plan, create a new GitHub repo or not, and how many questions to put in the bank (54, 108 or 180). Each had a "Recommended" option.
4. Click your answers and submit.

5. Cursor writes a **plan**: a Markdown document with the folder layout, the design decisions, diagrams and a to-do list. It opens in the editor.
6. **Read the plan.** This is your job. You can edit the plan file directly, like any document, or reply in the chat ("don't use UUIDs", "also deploy to Vercel after testing") and it updates the plan. I did this once: I asked it to add "test locally, push to GitHub, deploy on Vercel" to the frontend plan before building.
7. Click **Build**. Cursor switches to building, ticks off each to-do as it goes, and reports back at the end.

Plans are saved in your home folder by default, so they survive after the chat ends. If you want the plan inside your project (to share with teammates), click **Save to workspace**.

> **WHY BOTHER PLANNING?**
>
> My backend plan noticed something I hadn't thought about: Render's free tier wipes its disk on every restart, so saved interview sessions would vanish. It asked me before writing any code.

1.7 Give Cursor context with @

The agent can search your project on its own, but pointing it at the right files gets better results. Type `@` in the chat box and a menu pops up:

WHAT YOU TYPE	WHAT IT DOES	HOW I USED IT
`@prd.md`	Attaches one file	Gave Cursor the PRD for the backend
`@stitch_ai_interview_coach_webapp/`	Attaches a whole folder	Gave Cursor all the Stitch designs at once
Drag or paste an image	Attaches a screenshot	Great for "make it look like this" or "here's the error"
`@Chats`	Brings in an earlier conversation	Useful when a new chat needs to know what an old one did
`@Terminals`	Attaches what's in your terminal	"Why did this command fail?"
`@Browser`	Lets the agent use Cursor's built-in browser	"Open localhost:3000 and test the signup flow"
Paste a link	Points the agent at documentation	For libraries newer than the AI's training. (The old `@Docs` option was removed in 2026.)

A folder from outside your project works too. My designs were in `Downloads`, and I attached that folder directly.

1.8 Model picker: leave it on Auto

Next to the mode picker is the model picker (number 6). **Auto** lets Cursor pick a model for each request, balancing quality and cost. I used Auto for everything in this guide. Auto isn't included on every plan; if you don't see it, keep whatever model is selected by default. As a beginner you don't need to think about models at all.

1.9 Review changes, and how to undo

Cursor edits files directly, but you stay in control:

- **Changed files are listed** at the bottom of the agent panel after each turn. Click a file to see the diff: green lines added, red lines removed.
- **Keep or undo.** You can keep or undo (reject) changes per file, or all at once, using the buttons in that review area.
- **Checkpoints.** Hover over one of your earlier messages and you'll see a **Restore checkpoint** option. It rolls your files back to how they were at that point (the chat messages stay). Your safety net when the agent goes in the wrong direction. Checkpoints are separate from Git, so they also work before you've ever committed.

1.10 When the agent wants to run a command

The agent runs terminal commands all the time: `pip install`, `npm run build`, starting servers. Cursor decides which ones need your OK. The setting lives in **Cursor Settings** (`⌘ Cmd` + `Shift` + `J`) → **Agents** → **Approvals & Execution**:

- **Auto-review** (the default): safe commands run straight away, and risky ones are checked first or stopped for your approval.
- **Allowlist**: only commands you've approved before run without asking.
- **Run Everything**: no questions asked. Don't use this as a beginner.

When the agent does stop and show you a command, read it before you approve. Installing packages, running tests and starting servers are normal. Deleting files outside your project is not. Opening and clicking in the built-in browser also asks for approval by default.

1.11 Rules: instructions Cursor always follows

If you keep telling Cursor the same thing ("use TypeScript", "always write tests"), put it in a rule instead:

- **Project rules** live in your project, in a `.cursor/rules/` folder, or in a file called `AGENTS.md`. Our frontend has an `AGENTS.md` telling the agent to read the Next.js docs before writing code.
- **User rules** apply to all your projects. Set them under **Customize → Rules** in the sidebar.
- Easiest way to make one: type `/create-rule` in the chat and describe the rule.

You don't need rules to start. Add them when you notice yourself repeating something.

1.12 Shortcuts worth learning on day one

ACTION	MAC	WINDOWS
Open / close the Agent panel	`⌘` + `I`	`Ctrl` + `I`
New chat	`⌘` + `N`	`Ctrl` + `N`
Cycle modes (Agent → Plan → …)	`Shift` + `Tab`	`Shift` + `Tab`
Send selected code to the agent ("explain this")	select, then `⌘` + `L`	select, then `Ctrl` + `L`
Accept an autocomplete suggestion (grey text while typing); `Esc` rejects it	`Tab`	`Tab`
Voice input in the chat box	`⌘` + `Shift` + `Space`	`Ctrl` + `Shift` + `Space`
Command palette (search every command)	`⌘` + `Shift` + `P`	`Ctrl` + `Shift` + `P`
Find a file by name	`⌘` + `P`	`Ctrl` + `P`
Open / close the terminal	`Ctrl` + `` ` ``	`Ctrl` + `` ` ``
Cursor Settings	`⌘` + `Shift` + `J`	`Ctrl` + `Shift` + `J`

Older tutorials say `⌘` + `K` opens "inline edit". That changed in 2026: selecting code and pressing `⌘` + `K` or `⌘` + `L` now sends it to the agent instead. If a shortcut here doesn't match your Cursor, press `⌘` + `Shift` + `P` and search for the action by name.

TRY IT NOW (5 MINUTES)

Open your empty project folder in Cursor. Switch to **Ask** mode and type: *"I'm a complete beginner. What's the difference between a frontend and a backend? Use a restaurant analogy."* Then switch to **Agent** mode and type: *"Create a file called hello.md that says hello in 3 languages."* Watch the file appear in the Explorer. Then undo it. You've now used two modes and the undo.

PART 2

Research and the PRD

A **PRD** (Product Requirements Document) is a plain description of what you're building: who it's for, what screens it has, what data it stores, which technologies, and where it'll be hosted. It's the single most important input to Cursor. A vague PRD gets you a vague app.

I didn't write mine from scratch. I did the research in ChatGPT and had it produce the PRD.

2.1 Research in ChatGPT

1. Open chatgpt.com. If you have the option, turn on **Deep research** or web search so it looks at real products, not just its memory.
2. Ask it to research the problem and what already exists. Example prompt below.
3. Read what it gives back. Push back where it's generic ("which of these complaints come from Indian students specifically?").

EXAMPLE PROMPT · RESEARCH

I want to build a web app that helps CS students practice technical interviews with multiple-choice questions across DSA, system design, Python, Java, JavaScript, React, Next.js and FastAPI. Research what tools already exist, what students complain about, and what a simple first version (MVP) should include. Keep it practical: this is a student capstone project.

2.2 Turn the research into a PRD

In the same ChatGPT chat:

EXAMPLE PROMPT · PRD

Now write a detailed PRD in Markdown for the MVP. Include: product vision, problem statement, target users, every screen and what's on it, user flow, data model (tables and columns), API endpoints, folder structure for a monorepo, and tech stack: Next.js + TypeScript + Tailwind for the frontend, FastAPI + SQLAlchemy + SQLite for the backend. Frontend will be hosted on Vercel, backend on Render free tier, code on GitHub. No login for the MVP: the user just enters their name. Include a "future scope" section for AI-generated questions.

1. Copy the full Markdown output.
2. Open a plain-text editor, or create a new file in Cursor: **File → New File**, paste, save as `prd.md` inside your project folder.

My PRD had these sections, so check yours has something similar: project overview, target audience, features (name entry, subject selection, interview config, MCQ screen, results, review), data model, API list, folder structure, deployment, and future scope.

WATCH OUT

Say where you'll host things, **in the PRD**. Cursor made different choices because it knew we'd use Render's free tier, like reloading the question bank on every boot because the free disk gets wiped.

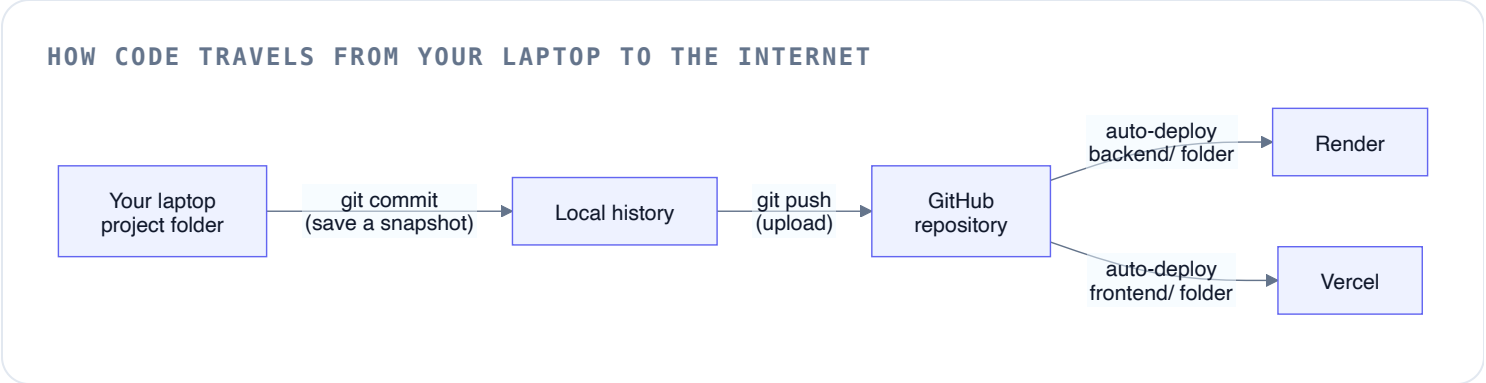
PART 3

Build the backend with Plan mode

I built the backend first and told Cursor to leave the frontend empty, because my designs weren't ready yet. You don't have to do it in this order, but it worked well: the designs could then be made to match the real API.

3.1 Two things to understand first: monorepo and GitHub

A **monorepo** is one project folder holding both the frontend and the backend. **GitHub** stores a copy of that folder online. Render and Vercel both watch the same GitHub repository: every time new code is pushed, they rebuild automatically.



You don't need to type git commands. Cursor runs them for you, as long as you did the `gh auth login` step in Part 0.

3.2 Send the backend prompt

1. Make sure `prd.md` is in your project folder and the folder is open in Cursor.
2. Open the Agent panel, click **+** for a new chat.
3. Set the mode picker to **Plan**.
4. Type the prompt below. Where it says `@prd.md`, type `@` and pick the file from the menu so it's attached properly.
5. Press `Enter`.

PROMPT I USED · PLAN MODE

Okay there is a PRD, I am attaching PRD.md file. It has all the requirement of what we are trying to build. For now, I want you to focus on building MVP, okay? You will not be touching the frontend. You will only work on the backend. Frontend, you just keep a placeholder, that's it. Nothing, no components, nothing should be built. The reason is I am getting the design done. Once the design are done, I will share you the screenshot and the design. After that, we will start building the frontend. So, we'll be using Monorepo, all the information is available in PRD file. Also, remember that we'll be hosting this on backend on render and frontend on Vercel, and we'll be pushing this into a GitHub repository. So, based on this I want you to start implementing, finish the backend and put the basic structure for frontend. Once I give you the design, you will start working on the design. @prd.md

Look at what's in there. Nothing technical. It says **what** (backend MVP), **what not** (no frontend components), **why** (designs are coming), and **where** it'll live (Render, Vercel, GitHub).

3.3 Answer the questions, read the plan, click Build

Plan mode came back with three questions. These were the exact options:

QUESTION CURSOR ASKED	WHAT I PICKED
Render's free tier wipes the disk on restart, so interview sessions get lost. Free tier, or paid ($7/month) with a persistent disk?	Free tier (recommended for a capstone)
How should I handle the GitHub repository?	Create a new repo with `gh` and push to it
How big should the question bank be? 54, 108 or 180 questions?	About 12 per subject, 108 total (recommended)

Then it wrote the plan, titled *"AI Interview Coach: Backend MVP and Monorepo Setup"*, with 10 to-dos: scaffold the monorepo, backend core, database models, question bank, services, API routes, tests, frontend placeholder, verify, and push to GitHub. It even included an architecture diagram.

One thing in the plan I'd never have thought of: it chose random IDs for interview sessions instead of 1, 2, 3, "so nobody can open other students' results by guessing the next number". It flagged that as a deliberate change from the PRD. That's the kind of thing to look for when you read a plan.

I read it and clicked **Build**. Eight minutes later:

```
octeveaiint/
├── backend/          FastAPI app: 8 API endpoints, 108 questions, 48 tests
│   ├── app/
│   │   ├── api/routes/     health, subjects, questions, interviews
│   │   ├── models/         database tables
│   │   ├── schemas/        what the API sends and receives
│   │   ├── services/       question picking, scoring
│   │   └── db/data/questions.json
│   ├── tests/
│   └── requirements.txt
├── frontend/         Next.js "coming soon" placeholder
├── docs/PRD.md       your PRD, moved here
├── render.yaml       instructions for Render (free plan)
└── README.md
```

It also ran all 48 tests, started the server, played through a full interview with test requests, created a **private** GitHub repository, and pushed the code. I didn't open GitHub once.

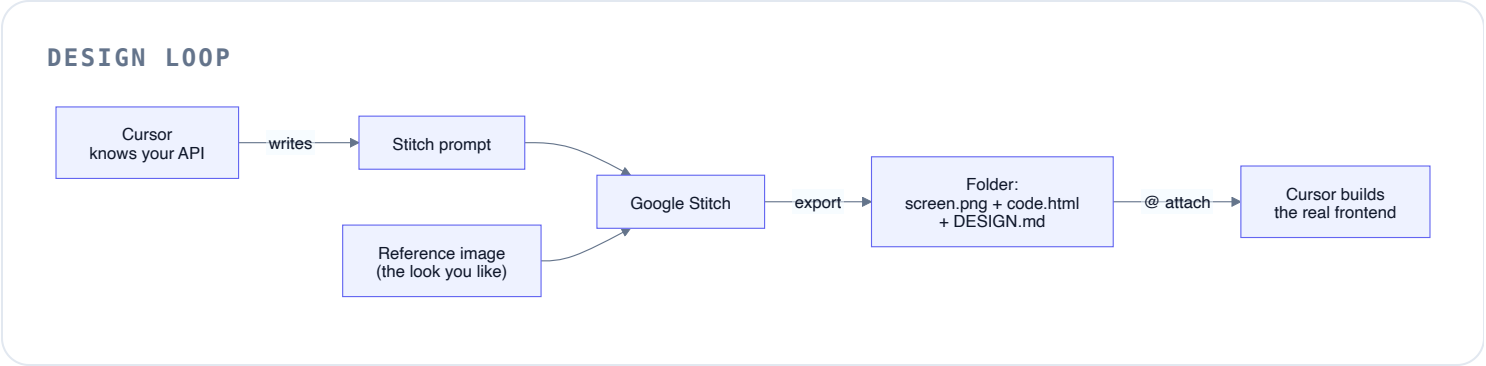
DON'T SKIP READING THE SUMMARY

At the end, Cursor writes a summary with a "Things to know" section. Mine said: "The frontend needs Node 22. Your default Node, v20.11, fails the build." That one line would have cost me an hour of confusion later. Read the whole summary every time.

PART 4

Design the screens with Google Stitch

While the backend was building, I opened a second chat to work on the design. Google Stitch is an AI tool that turns a text description (and reference images) into screen designs and HTML code.



4.1 Let Cursor write the Stitch prompt

Why ask Cursor and not write it yourself? Cursor has read your backend. It knows the exact screens, the fields the API returns, the error cases. So the designs come out matching what your backend can actually do.

1. Click **+** for a new chat. Keep the mode on **Agent** (or **Ask**; it only needs to read).
2. Send this:

PROMPT I USED

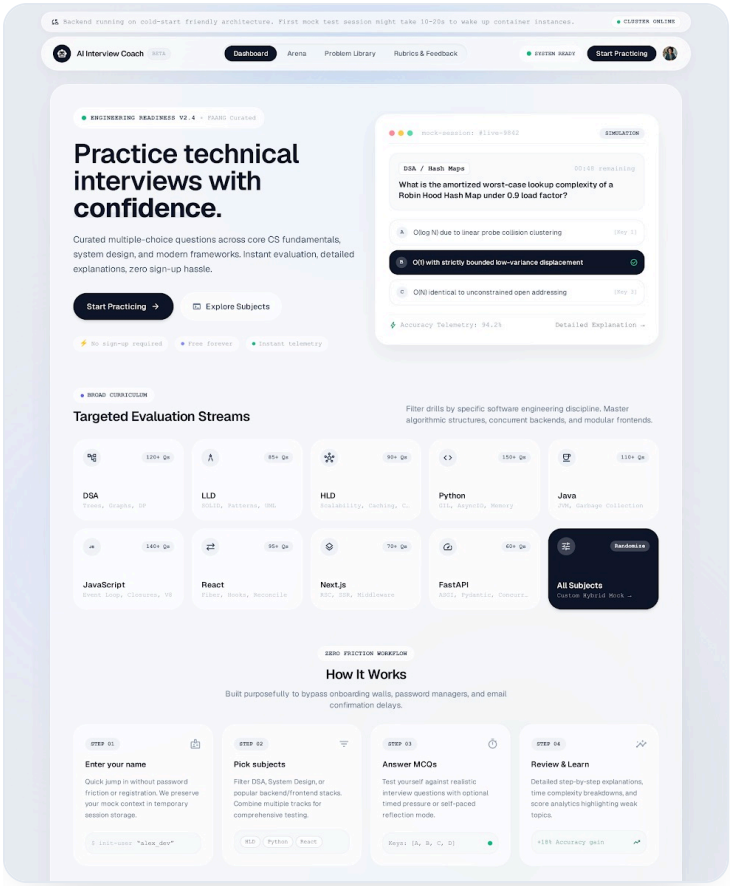
Now you have the idea what we are trying to build. So now I want you to generate a prompt which I will be giving to Stitch for building an AI interviewer frontend. So this will help it to build the frontend. You have the idea what we are building backend, you know all the APIs and everything. So give me a prompt which I can give Google Stitch to design my frontend.

It came back with a long prompt (about 1,300 words) describing all 8 screens, every loading and error state, sample data, and the design style. The full prompt is in the appendix so you can reuse it.

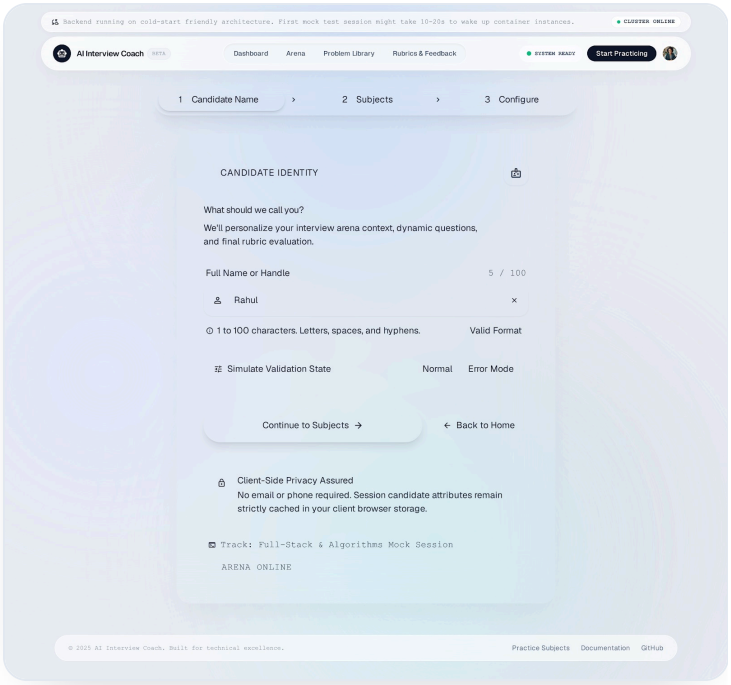
4.2 Generate the designs in Stitch

1. Go to stitch.withgoogle.com and sign in with your Google account.
2. Choose the **web** app option (not mobile) when it asks.
3. Find a **reference image** of a look you like. I wanted glassmorphism (frosted-glass panels, soft blurred colours), so I used a screenshot of a design in that style. Pinterest, Dribbble and Mobbin are good places to look.
4. Upload the image with the image/attach button in Stitch's prompt box.
5. Paste the prompt from Cursor and add one line about the image, like *"Use the attached image as the visual style: glassmorphism."*
6. Generate. Stitch creates the screens. You can ask for changes in the same box ("make the buttons black", "add a step indicator").
7. When you're happy, select the screens and **export / download** them. You get a folder per screen with a `screen.png` (the picture) and a `code.html` (the HTML), plus a `DESIGN.md` with the colours, fonts and spacing.
8. Unzip it somewhere you can find it. Mine was `Downloads/stitch_ai_interview_coach_webapp`.

These are the screens Stitch gave me:



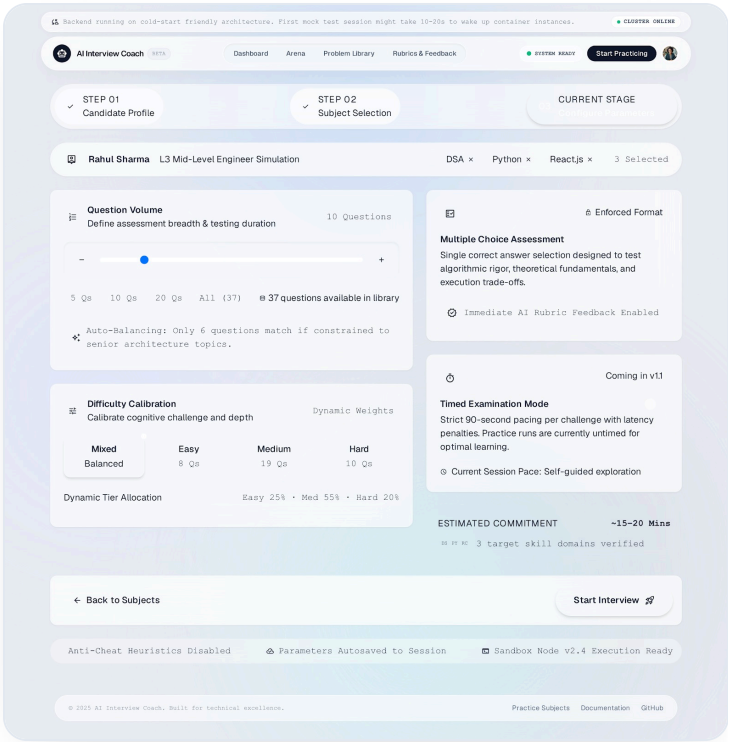
Landing page



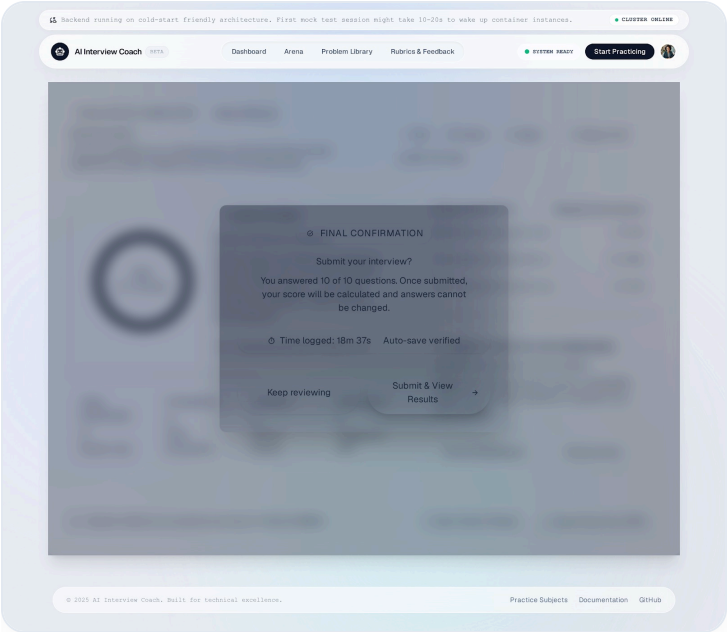
Step 1: your name



Step 2: choose subjects



Step 3: configure interview



Submit confirmation

Notice what's missing: no screen for answering questions, no results page, no answer review. Stitch didn't make them. Hold that thought; it comes back in Part 9.

PART 5

Run it on your laptop

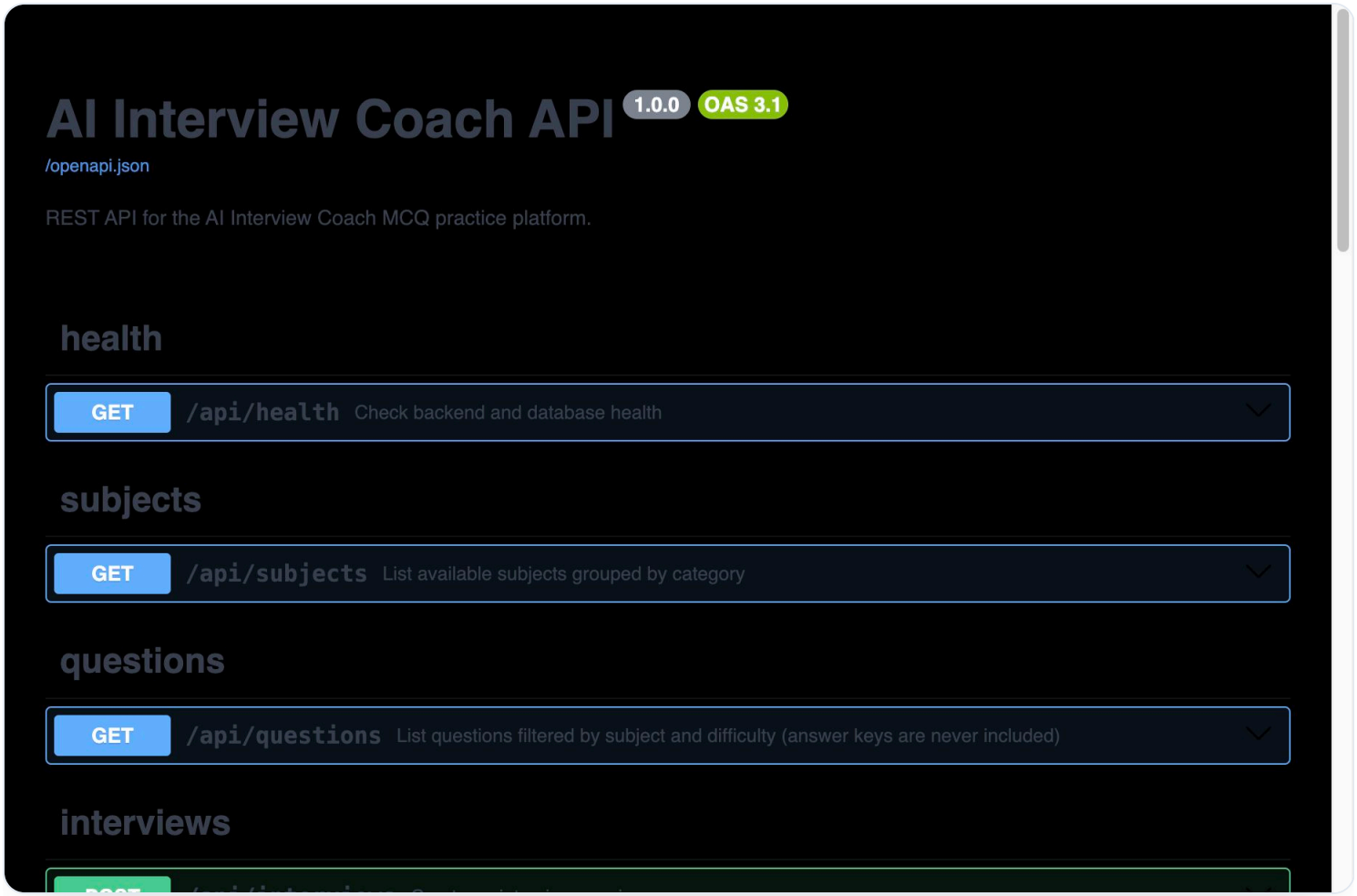
Before putting anything on the internet, run it on your own machine. "Localhost" means "this computer".

PROMPT I USED · AGENT MODE

Can you run this application for me and I want to test it locally.

Cursor created the settings files, started the backend on port 8000 and the frontend on port 3000, and gave me these links:

LINK	WHAT IT IS
`http://localhost:8000/docs`	The backend's test page. Every API endpoint, with a button to try it.
`http://localhost:3000`	The frontend. At this stage just "coming soon".



The `/docs` page. FastAPI makes it automatically. This is the live one, but your local one looks the same.

5.1 Play through an interview without a frontend

1. Open `http://localhost:8000/docs` in your browser.
2. Click **GET /api/subjects**, then **Try it out**, then **Execute**. You'll see the 9 subjects come back.
3. Click **POST /api/interviews** → **Try it out**. Replace the example body with:

   ```
   { "student_name": "Rahul", "subjects": ["DSA", "PYTHON"], "num_questions": 5,
   ```

   Click **Execute**. Copy the `id` from the response. Notice the questions come back **without** the correct answers.
4. Try **GET /api/interviews/{id}/results** with that id. You get an error (409), because you haven't submitted yet. That's on purpose.

5 Submit with **POST /api/interviews/{id}/submit**, then look at **/results** and **/review**.

TRY IT YOURSELF

Don't understand a response? Switch to **Ask** mode and paste it: *"What does this 409 error mean and why did the backend send it?"*

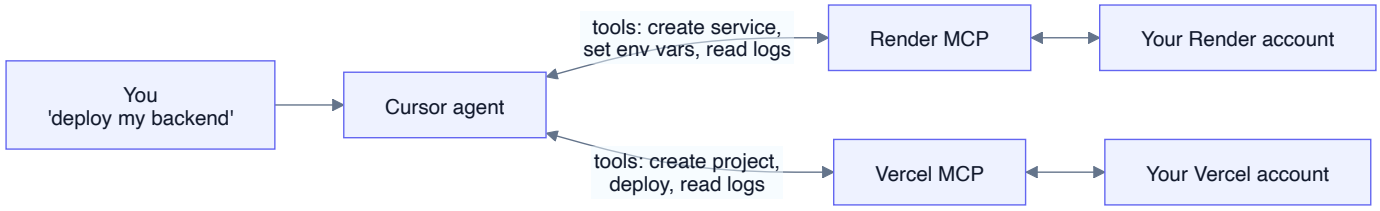
PART 6

Connectors: let Cursor control Render and Vercel

Out of the box, Cursor can edit files and run commands on your laptop. It can't log into your Render or Vercel account. A **connector** fixes that.

The technical name is **MCP** (Model Context Protocol). An MCP server is a small bridge that gives Cursor a set of tools for one service, like "create a web service on Render" or "read the Vercel build logs". In Cursor, these come packaged as **plugins**.

WHAT A CONNECTOR DOES



Honestly, I first asked Cursor to deploy with no connector. It opened the Render website in its built-in browser and asked me to log in so it could click through. Then I asked:

PROMPT I USED

Can we use some connector or MCP of render dot com available to host it?

Cursor checked Render's docs and told me how to add it. Here are the steps.

6.1 Add the Render connector

1. In the Cursor chat box, type `/add-plugin render` and press `Enter`.
2. A small window appears. Keep the default choice and click **Add plugin**.
3. Sign in when Cursor asks (the button says **Authenticate** or **Connect**). If it doesn't ask: click **Customize** in the sidebar → **MCPs**, find **Render**, and click its sign-in button there.
4. A Render sign-in page opens in your browser. Click **GitHub**, log in, and click **Authorize**. This

also creates a free Render account if you don't have one.

5. Back in Cursor, Render should show as connected (a green dot or a list of tools).

6.2 Add the Vercel connector

Same steps, with Vercel:

1. Type `/add-plugin vercel` and press `Enter`.
2. Click **Add plugin**, then **Authenticate**.
3. Sign in to Vercel in the browser (**Continue with GitHub**) and approve access.

Then tell Cursor you're done:

PROMPT I USED

I am done please help me deploy both frontend and backend

READ THIS BEFORE YOU CONNECT ANYTHING

A connector gives Cursor real control of that account: it can create services, change settings, redeploy. That's why it's useful, and also why you should only connect accounts you're OK with the agent touching. Cursor told me this itself before I connected. You can remove a connector any time from **Customize** in the sidebar.

If `/add-plugin` doesn't work in your version, there are two other ways in:

- Click **Customize** in the sidebar, search for Render or Vercel, click **Install**, and pick **user** scope so it works in all your projects.
- Browse cursor.com/marketplace. Render and Vercel are both official plugins there.

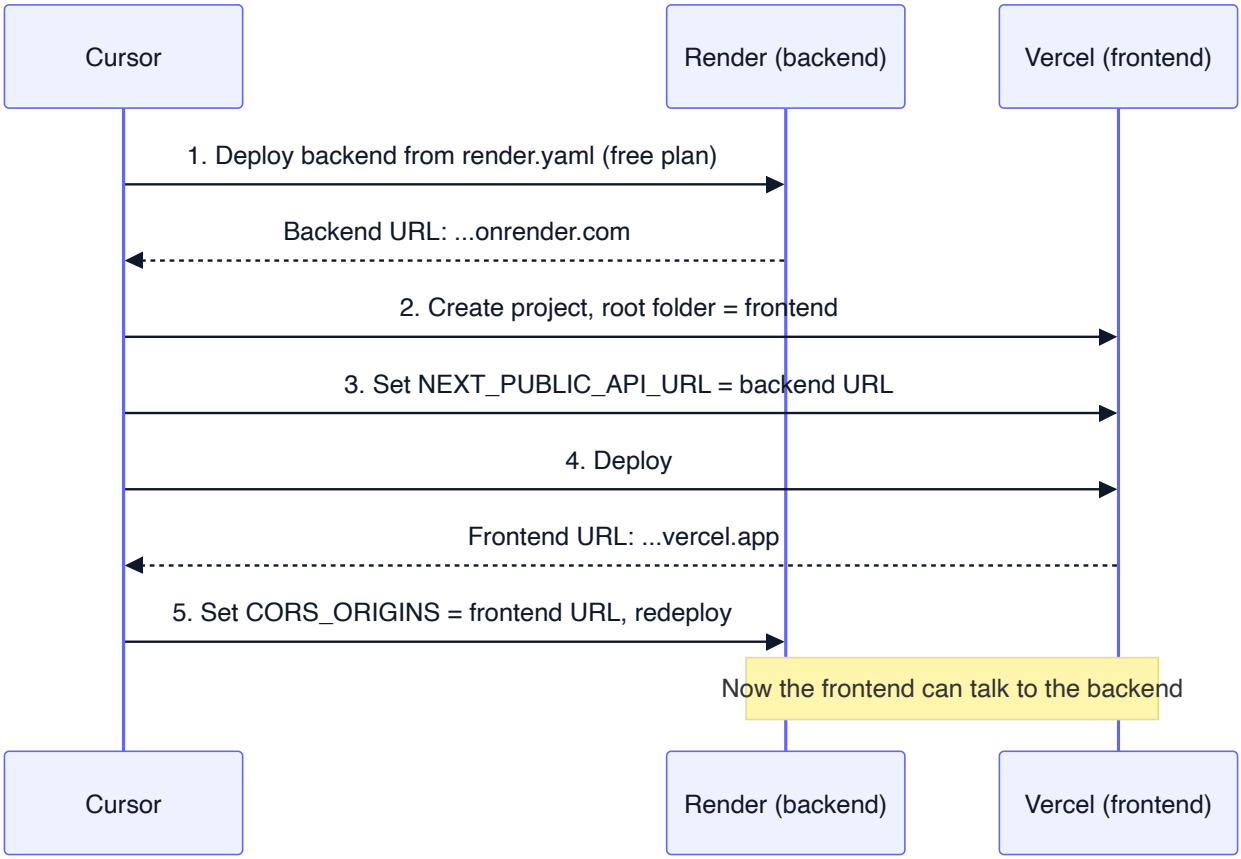
If a connector shows errors, open the Output panel (`⌘ Cmd` + `Shift` + `U`) and pick **MCP Logs** from the dropdown. Or just ask the agent: *"The Render connector isn't working. Check the MCP logs and tell me why."*

PART 7

Put the backend on Render

Order matters here, and it confuses everyone the first time, so look at this diagram slowly. The frontend needs the backend's address. The backend needs the frontend's address (to allow it in). So one has to go first.

THE ORDER OF DEPLOYMENT (THE CHICKEN-AND-EGG PROBLEM)



7.1 What Cursor did on Render

With the connector, I didn't click anything on Render's site. Cursor:

1. Listed my Render workspaces (I had one, "My Workspace") and checked it was empty.
2. Created a **Blueprint** from the `render.yaml` file in the repo. A blueprint is a recipe: "this is a Python web service, the code is in `backend/`, install with this command, start with that command, use the **free** plan".

3. Checked that the blueprint would only create one free service, then approved it.
4. Waited for the build (2 to 4 minutes), checked the health endpoint, and confirmed all 9 subjects came back.

My backend went live at `https://ai-interview-coach-api-dz7u.onrender.com`.

WHY ONLY THE BLUEPRINT ROUTE IS FREE

Cursor found that the connector's simple "create web service" tool only offers paid plans (cheapest about $7/month). The blueprint route reads `render.yaml`, which says `plan: free`. If you ask for free hosting, say so clearly in your prompt, like I did: "it has to be on the free tier".

7.2 Doing it by hand instead (if your connector doesn't work)

1. Log in at dashboard.render.com.
2. Click **New +** → **Blueprint**.
3. Connect your GitHub account if asked, and give Render access to your repository.
4. Pick your repository. Render finds `render.yaml` and shows the service it will create. Check it says **Free**.
5. It asks for the values of `CORS_ORIGINS` and `CORS_ORIGIN_REGEX`. For now put `http://localhost:3000` in `CORS_ORIGINS` and leave the other empty.
6. Click **Apply** / **Deploy Blueprint**. Wait for the status to turn **Live**, then open `your-url.onrender.com/api/health`.

FREE PLAN FACTS YOU MUST KNOW

- **It falls asleep.** After 15 minutes with no visitors, a free Render service shuts down. The next visitor waits about a minute while it wakes up. Open your site a minute before a demo.
- **It forgets.** The free disk is wiped on every restart or redeploy. Our app reloads its 108 questions on every boot, so questions are safe, but completed interview sessions disappear. To keep them, you'd need a paid disk or a real database like PostgreSQL.

PART 8

Put the frontend on Vercel

This is where the connector hit a wall, and where you'll see the three human moments.

8.1 What happened

1. Cursor checked which Vercel account the connector was logged into (it was my other email) and tried to create the project.
2. Vercel said no: the connector was signed in but didn't have access to my personal project space. Cursor generated an approval link, opened it in the browser panel inside Cursor, and asked me to log in to Vercel and click **Approve** within 10 minutes.
3. I logged in, approved, and typed: *"vercel is connected deploy again"*.
4. Cursor created the Vercel project with **Root Directory** set to `frontend`, and added the environment variable `NEXT_PUBLIC_API_URL` = my Render URL.
5. It noticed two settings that would have bitten me: Vercel's login protection was on for *all* deployments (students would have seen a Vercel login page instead of my site), and Node.js was set to a different version than the one it had tested on. It switched protection to **Standard** and Node to **22**.
6. It deployed, confirmed `octeveaiint.vercel.app` showed *my* app (not someone else's with the same name), then went back to Render and set `CORS_ORIGINS` to the Vercel URL and redeployed.
7. Finally it ran a full test interview against the live backend, pretending to be the Vercel site.

THE THREE HUMAN MOMENTS

Remember I asked you to keep count? Here they are. Every one was a **login or a permission**:

1. Clicking **Authenticate** and logging in for the Render connector.
2. Clicking **Authenticate** and logging in for the Vercel connector.
3. Logging in to Vercel in the browser panel and clicking **Approve** on the team access page.

Cursor can't type your passwords, and you shouldn't give them to it.

8.2 Doing it by hand instead

1. Log in at vercel.com. Click **Add New...** → **Project**.
2. Under **Import Git Repository**, find your repo and click **Import**. If it's not listed, click to adjust GitHub app permissions and give Vercel access to that repo.
3. **Root Directory:** click **Edit** and choose `frontend`. This is the step everyone forgets in a monorepo. Vercel then detects Next.js on its own.
4. Open **Environment Variables**. Name: `NEXT_PUBLIC_API_URL`. Value: your Render URL, like `https://your-api.onrender.com` (no slash at the end).
5. Click **Deploy**. Wait 1 to 2 minutes.
6. Copy your `.vercel.app` URL. Go back to Render → your service → **Environment**, set `CORS_ORIGINS` to that URL, save. Render redeploys.
7. If your site shows a Vercel login page to other people: Vercel project → **Settings** → **Deployment Protection** → set Vercel Authentication to **Standard Protection**.

WHAT IS CORS, IN ONE PARAGRAPH

Browsers block a website from calling a backend on a different address unless the backend says "I allow that website". `CORS_ORIGINS` is the backend's guest list. If your live site loads but shows "can't reach server" or nothing loads, the guest list is the first thing to check.

After this, updates are automatic. Every time Cursor pushes to GitHub, Render and Vercel rebuild within a few minutes.

PART 9

Build the real frontend from the designs

The internet now had a working backend and a "coming soon" page. Time to build the actual screens.

9.1 Attach the design folder and ask for a plan

1. New chat (**+**). Mode: **Plan**.
2. Type `@` and attach the Stitch folder. If it's outside your project, you can type its path, like `@/Users/you/Downloads/stitch_ai_interview_coach_webapp/`, or drag the folder into the chat box.
3. Send:

PROMPT I USED · PLAN MODE

@/Users/prithvirajpillai/Downloads/stitch_ai_interview_coach_webapp/ Now this folder contains the inspirations and how I want to build the page. It also has the HTML file and the PNG files. I want you to use this as a reference. The idea is we'll be building the frontend using Glassmorphism design. If you feel that a page is missing, always create reusable components. Do not create the same component again and again in different pages. Use the reusable components in everything, and then you start building that.

"If a page is missing" is the line that handles the three screens Stitch didn't make (answering questions, results, review). Cursor designed those by extending the same style.

"Reusable components" is the line that keeps the code clean. Instead of copying the same button into seven pages, Cursor made one `Button`, one `GlassPanel`, one `Modal`, and reused them everywhere.

9.2 The question it asked me

Plan mode spotted that the Stitch designs had fake stuff in them: nav links to pages that don't exist ("Arena", "Problem Library"), a profile picture, status text like "Cluster Online" and "Anti-Cheat Heuristics Disabled". It asked:

OPTION	
Keep the visual style, but only show elements that actually work. Drop the fake telemetry and toggles. (Recommended)	**I picked this**
Match the mockups pixel-for-pixel, including the fake elements	
Pare it down further: glass style only	

WATCH FOR THIS IN YOUR DESIGNS

If I'd picked "pixel-for-pixel", the live site would have a "Problem Library" link that goes nowhere and a status strip saying "Anti-Cheat Heuristics Disabled". Look through your own Stitch screens for things like that before you build.

9.3 Add "test, push, deploy" to the plan

Before clicking Build, I sent one more message so the plan would include shipping:

PROMPT I USED · STILL IN PLAN MODE

Okay, now once you've done this, test it locally and see how it is working. If it's working fine, then I want you to deploy the, push the changes to GitHub repository and deploy it on Vercel also.

Cursor checked that GitHub, Vercel and Render were already connected, and added two to-dos: push only after local testing passes, then watch the Vercel deployment and test the live site. The final plan had 10 to-dos. I clicked **Build**.

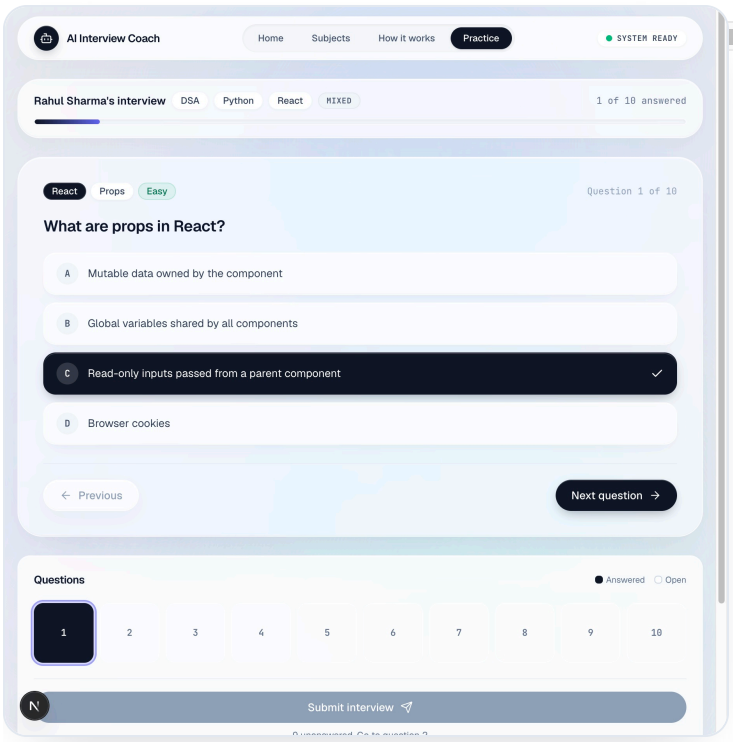
9.4 What it built, and how it tested

About 17 minutes later, all 7 screens were built from 47 components, organised like this:

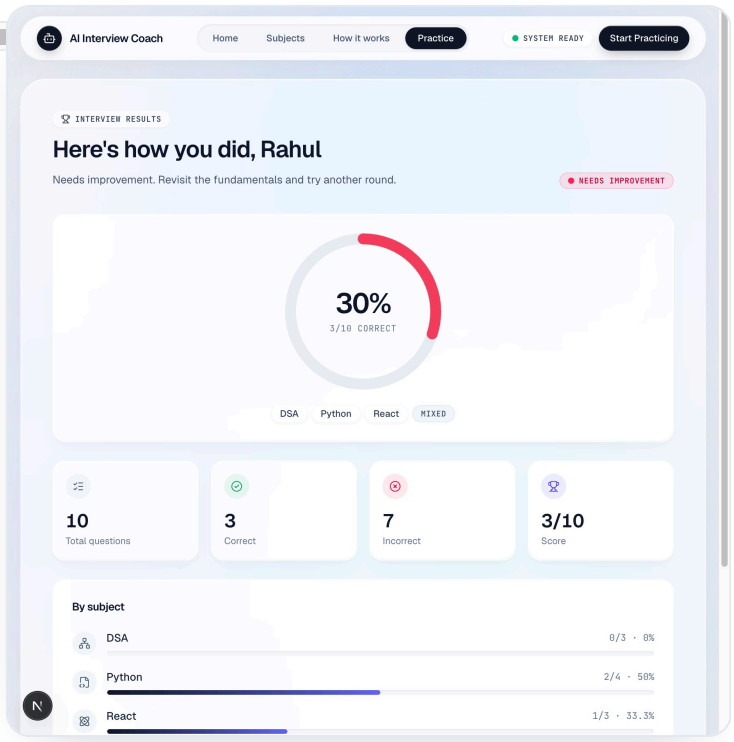
```
frontend/components/
├── ui/          Button, GlassPanel, Badge, Modal, TextField, ProgressBar … (15 build
├── layout/      SiteHeader, SiteFooter, WizardStepper, ColdStartBanner …
├── landing/     Hero, HowItWorks, SubjectGrid …
├── setup/       NameStep, SubjectsStep, ConfigureStep
```

```
├── interview/   QuestionCard, OptionRow, QuestionNavigator, SubmitConfirmModal …
└── results/     ScoreRing, StatTile, ReviewItemCard …
```

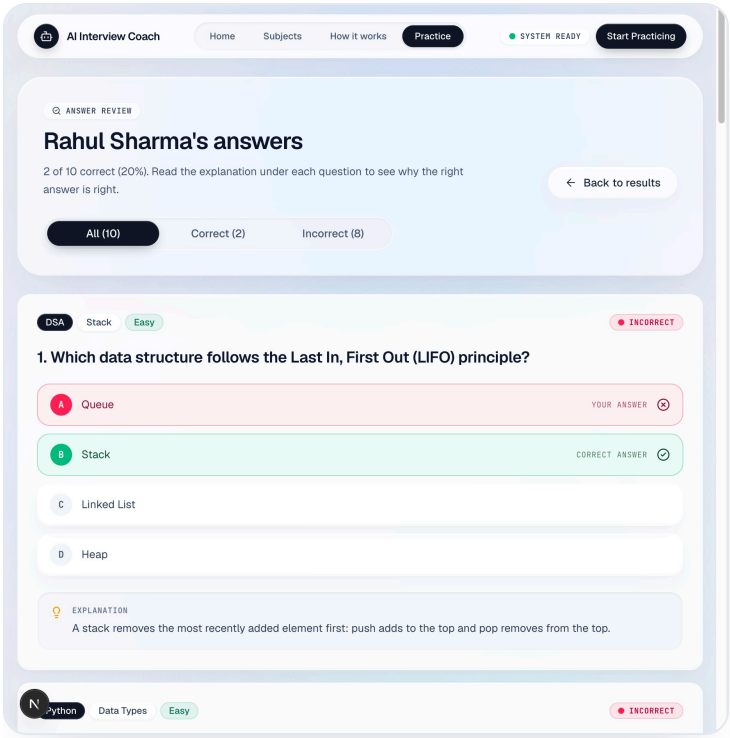
Then Cursor opened its own **built-in browser**, typed a name, picked subjects, answered questions, submitted, and checked the results and review pages, on desktop and at phone width. Here are the three screens Stitch never designed:



Answering questions



Results



Answer review

IT FOUND THREE BUGS ON ITS OWN

- **No blur anywhere.** The whole point of glassmorphism, missing. A CSS line written for old Safari was cancelling the blur in Chrome.
- **Results page frozen after submitting.** The "are you sure?" popup stayed open invisibly and blocked every click.
- **Bottom bar hard to read.** Content showed through it, so it made it more solid.

All three were caught in the browser and fixed *before* the push, because prompt 10 said "test it locally" first.

Then it pushed to GitHub, Vercel rebuilt automatically, and Cursor repeated the whole click-through on the live site.

PART 10

Add AI with Gemini

Up to here, the "AI Interview Coach" had no AI in it. The questions came from a fixed bank of 108. This was the next step, and I was still in the middle of it when I wrote this guide, so treat this Part as the plan, not the finished result.

10.1 Get a Gemini API key

1. Go to aistudio.google.com and sign in with Google.
2. Click **Get API key** → **Create API key**.
3. Copy it and keep it somewhere private. **Never** paste it into a public chat, a screenshot, or a file that gets pushed to GitHub.

10.2 The prompt

PROMPT I USED · PLAN MODE

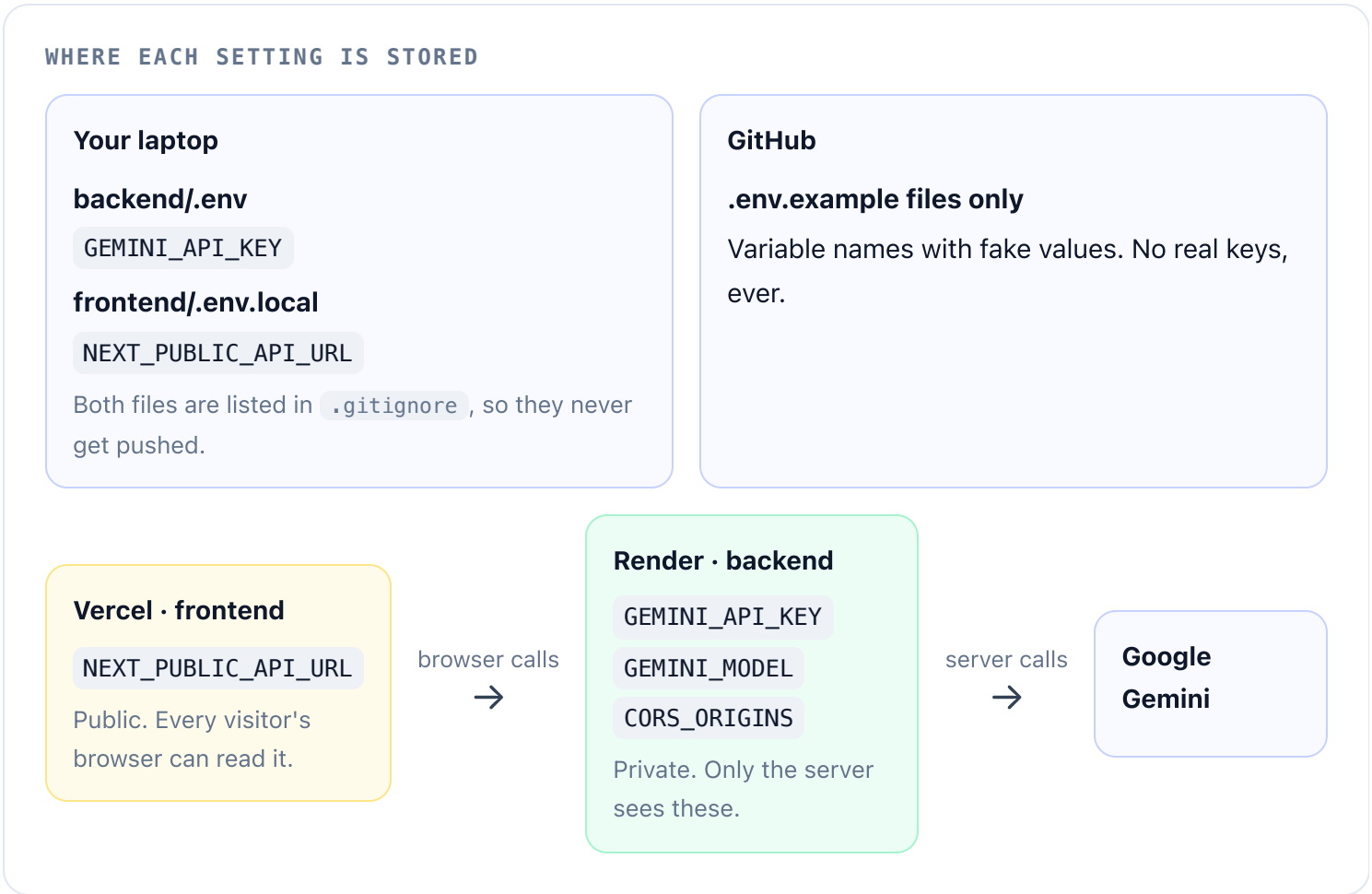
> Okay, now this is great, everything is working fine. Now what I want you to do is, I want you to implement AI into this, right? That was the step which is missing. So basically, as ever, user selects the question, response figures, and generates question, the question should be generated using AI. And after user answers the evaluation also should be done using AI. For this, I will be using Gemini API key, and the model will be Gemini three point one flash lite. So based on this, start making the changes, and I also want you to handle the environment key, which is required to connect both my frontend and backend, and in my backend also we need to give Gemini API key. I'll give that, I want you to take care of that into the deployment also.

Plan mode looked up the exact model name in Google's docs (`gemini-3.1-flash-lite`) and asked three questions:

QUESTION	RECOMMENDED ANSWER
What should "AI evaluation" mean? MCQs can be scored exactly without AI.	Keep MCQs. Gemini generates the questions. Scoring stays exact. After submitting, Gemini writes a personal report: why each wrong pick was wrong, weak topics, what to study next.
What if Gemini is down or out of quota?	Fall back to the 108-question bank, with a small note that AI was unavailable.
How do you want to give the API key for Render?	Add it yourself in the Render dashboard, so the key never goes in the chat.

10.3 Where secrets live

This diagram is the most important security lesson in the guide:



THE RULE

Anything starting with `NEXT_PUBLIC_` is sent to every visitor's browser. Anyone can read it. So the Gemini key goes **only** on the backend (your `backend/.env` and Render's Environment page), never on Vercel and never with a `NEXT_PUBLIC_` name. The browser asks your backend; your backend asks Gemini.

To add the key on Render by hand: Render dashboard → your service → **Environment** → **Add Environment Variable** → key `GEMINI_API_KEY`, value: your key → **Save Changes**. Render redeploys with it.

What I don't know yet: how well a free Gemini quota holds up with a full class of you hitting it at the same time. That's what the bank fallback is for. We'll find out.

REFERENCE

What went wrong (and the fix)

Every one of these really happened during this build.

WHAT YOU SEE	WHY	FIX
Frontend build fails locally with an error from Tailwind or Turbopack	Node.js version too old. I had v20.11; the build needed 20.19+.	Install Node 22. Tell Cursor: *"Check my Node version and pin the project to Node 22."*
Cursor started a server, but the page says "can't connect"	The server died when the command that started it finished.	Ask Cursor to run it as a background process. It caught and fixed this itself.
Vercel connector: "no access to team / scope"	The connector was logged in, but not approved for your project space.	Let Cursor generate the approval link, log in, click **Approve** within 10 minutes.
Your live site shows a Vercel login page to others	Deployment Protection set to all deployments	Vercel → Settings → Deployment Protection → Standard Protection
Site loads, but subjects never appear	Either the backend is waking up (wait 1 minute), or CORS is missing your Vercel URL	Open `your-api.onrender.com/api/health`. If that works, check `CORS_ORIGINS` on Render.
Past interview results vanished	Render's free disk was wiped on restart	Expected on the free plan. Paid disk or PostgreSQL if you need them kept.
Glass panels have no blur in Chrome	Old Safari-only CSS overriding the standard one	Cursor found it by testing in the browser. Ask: *"Test the site in the browser and check that backdrop blur works."*
You don't know what's wrong at all		Switch to **Debug** mode, describe what you see, paste the error.

REFERENCE

Every prompt I typed, in order

This is the complete list, copied from my chat history, lightly cleaned of typos. Adapt them for your own project.

#	MODE	PROMPT (SHORT VERSION)	PART
1	Plan	"There is a PRD... only work on the backend... frontend just a placeholder... monorepo... Render, Vercel, GitHub. @prd.md"	3
2	Plan	Answered 3 questions, read the plan, clicked **Build**	3
3	Agent	"Generate a prompt which I will be giving to Stitch... you know all the APIs."	4
4	Agent	"Can you run this application for me and I want to test it locally."	5
5	Agent	"Deploy my application on render.com on free tier... I am a non-technical person."	6
6	Agent	"Can we use some connector or MCP of render dot com available to host it?"	6
7	Agent	"I am done please help me deploy both frontend and backend"	7
8	Agent	"vercel is connected deploy again"	8
9	Plan	"@stitch folder... Glassmorphism... if a page is missing... reusable components."	9
10	Plan	"Test it locally... push the changes to GitHub... deploy it on Vercel." Then **Build**.	9
11	Plan	"Implement AI... Gemini... 3.1 Flash Lite... handle the environment keys... deployment."	10

YOUR GUESS FROM THE START

How many of these prompts contain code? **Zero.** Now go back and reread prompt 1.

▶ **Appendix: the full Stitch prompt Cursor wrote (click to open)**

REFERENCE

Glossary

WORD	MEANING
Frontend	The part people see and click: pages, buttons, forms. Ours is Next.js on Vercel.
Backend	The part that stores data and does the logic, like scoring. Ours is FastAPI (Python) on Render.
API / endpoint	The backend's menu. Each endpoint is one thing you can ask for, like `GET /api/subjects`.
Database	Where data is saved. Ours is SQLite, a single file.
PRD	Product Requirements Document. What to build and why.
MVP	Minimum Viable Product. The smallest version that actually works.
Monorepo	One repository holding several apps, here `backend/` and `frontend/`.
Repository (repo)	A project folder tracked by Git, usually also stored on GitHub.
Commit / push	Commit: save a snapshot of your code. Push: upload your commits to GitHub.
Terminal	A text window where you type commands instead of clicking.
localhost / port	localhost is your own computer. The port is like a door number: 8000 for our backend, 3000 for the frontend.
Deploy	Put your app on a server so anyone on the internet can use it.
Environment variable	A setting kept outside the code, like an API key or a URL, that changes between your laptop and the live server.
.env file	A file holding environment variables on your laptop. Never pushed to GitHub.
API key	A password that lets your app use a service like Gemini. Keep it secret.
CORS	The backend's guest list of websites allowed to call it from a browser.

MCP / connector / plugin	A bridge that gives Cursor tools to work with another service, like Render or Vercel.
Blueprint (`render.yaml`)	A file describing how Render should build and run your service.
Cold start	The wait while a sleeping free server wakes up.
Glassmorphism	A design style with frosted-glass, semi-transparent, blurred panels.
Component	A reusable piece of UI, like a Button, built once and used on many pages.

YOUR TURN

Homework

Don't rebuild my app. Build a different one with the same pipeline. Some ideas: a quiz app for your college's aptitude round, a flashcard app for a subject you're failing, a mock HR-interview app for non-CS friends.

- [ ] A `prd.md` written with ChatGPT, saved in your project folder
- [ ] A backend built with **Plan mode**. Screenshot the questions Plan mode asked you
- [ ] Stitch designs made from a Cursor-written prompt and your own reference image
- [ ] The frontend built from those designs, with reusable components
- [ ] Render and Vercel connected as connectors in Cursor
- [ ] Both deployed on free plans, with the frontend talking to the backend

Bring two things to the next class: your live `.vercel.app` link, and the one bug that cost you the most time. We'll go through the bugs first.